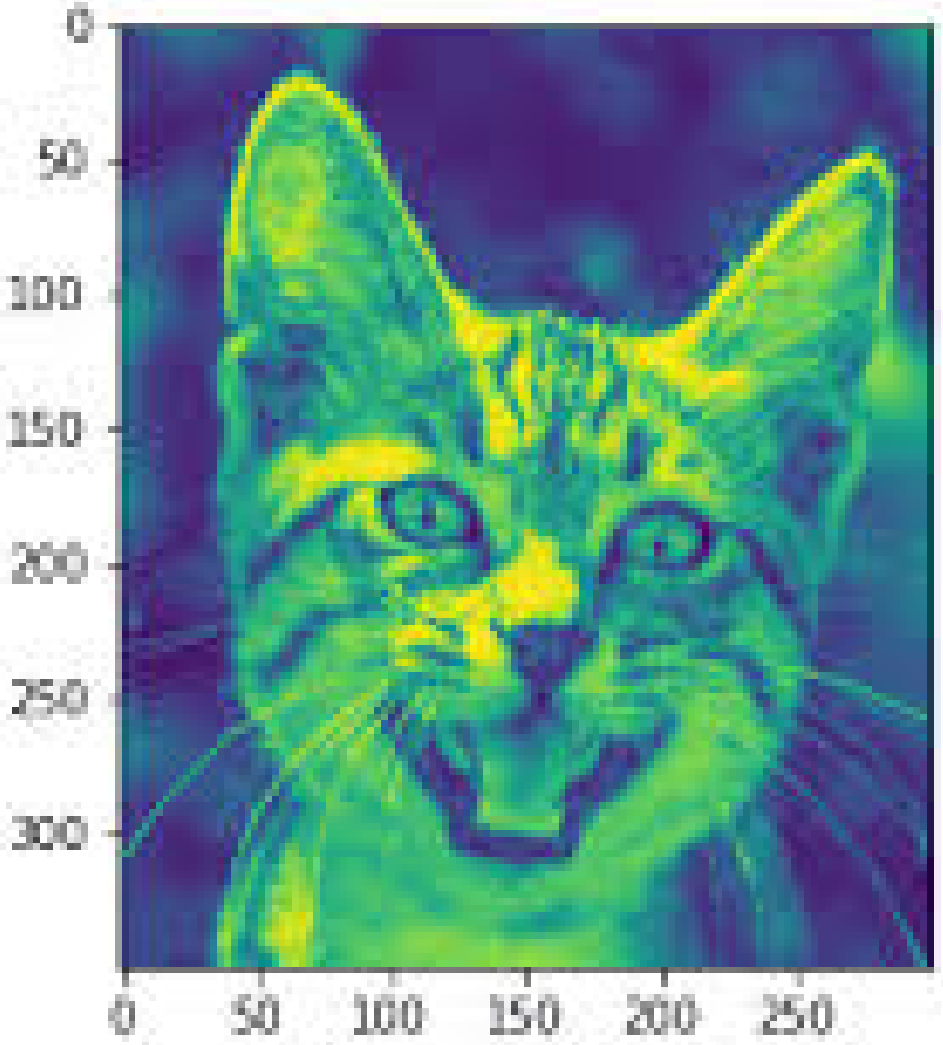

0
50
100
150
200
250
300
0
50
100
150
200
250

Глава 11. Память. DMA & FLASH

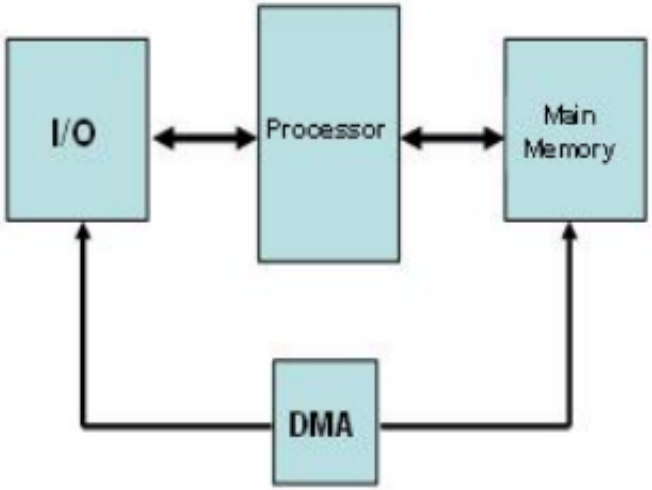
Ежелев Г. И.

18 апреля 2026 г.

DMA

DMA - Direct Memory Access - контроллер прямого доступа к памяти(ПДП).

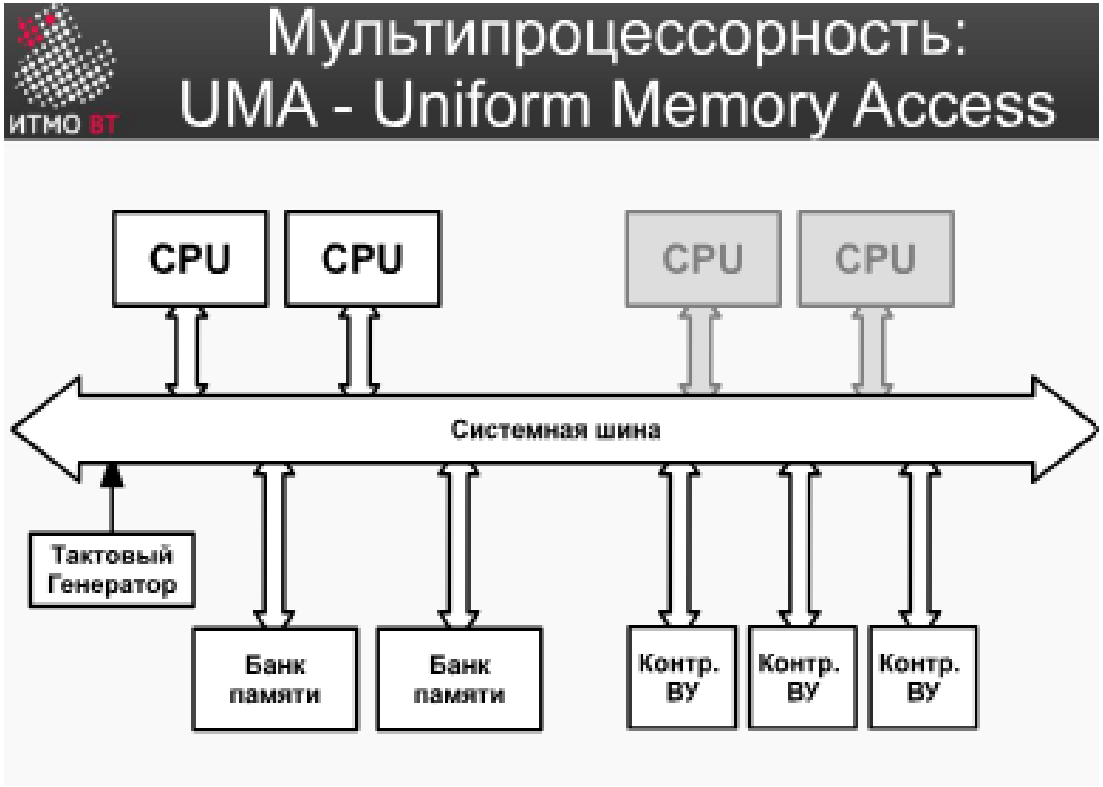
- Что это?
  - Прямой доступ к памяти - это третий из способов взаимодействия с периферией (После прямого чтения и прерываний).
  - Его задача еще больше сократить время взаимодействия процессора с периферией.
  - DMA контроллер напрямую пересылает данные из периферических устройств в память, минуя CPU.



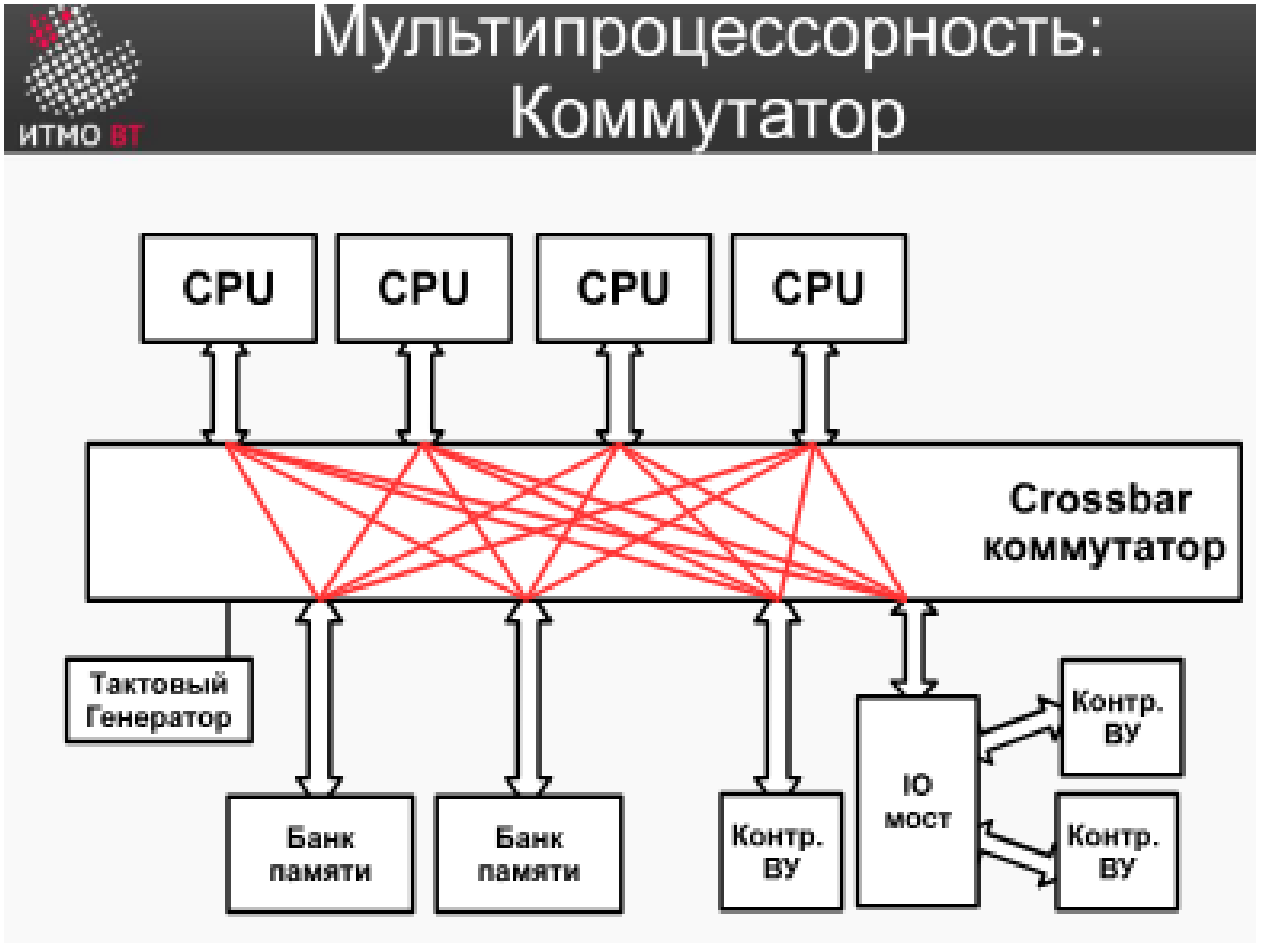
- Отсюда и название - прямой доступ к памяти - ведь по умолчанию все периферические устройства являются ведомыми и не могут сами обращаться к памяти МК (расположенной к слову на тех же шинах AHB, проверьте это на 19 странице datasheet!)

Отступление. Crossbar коммутатор

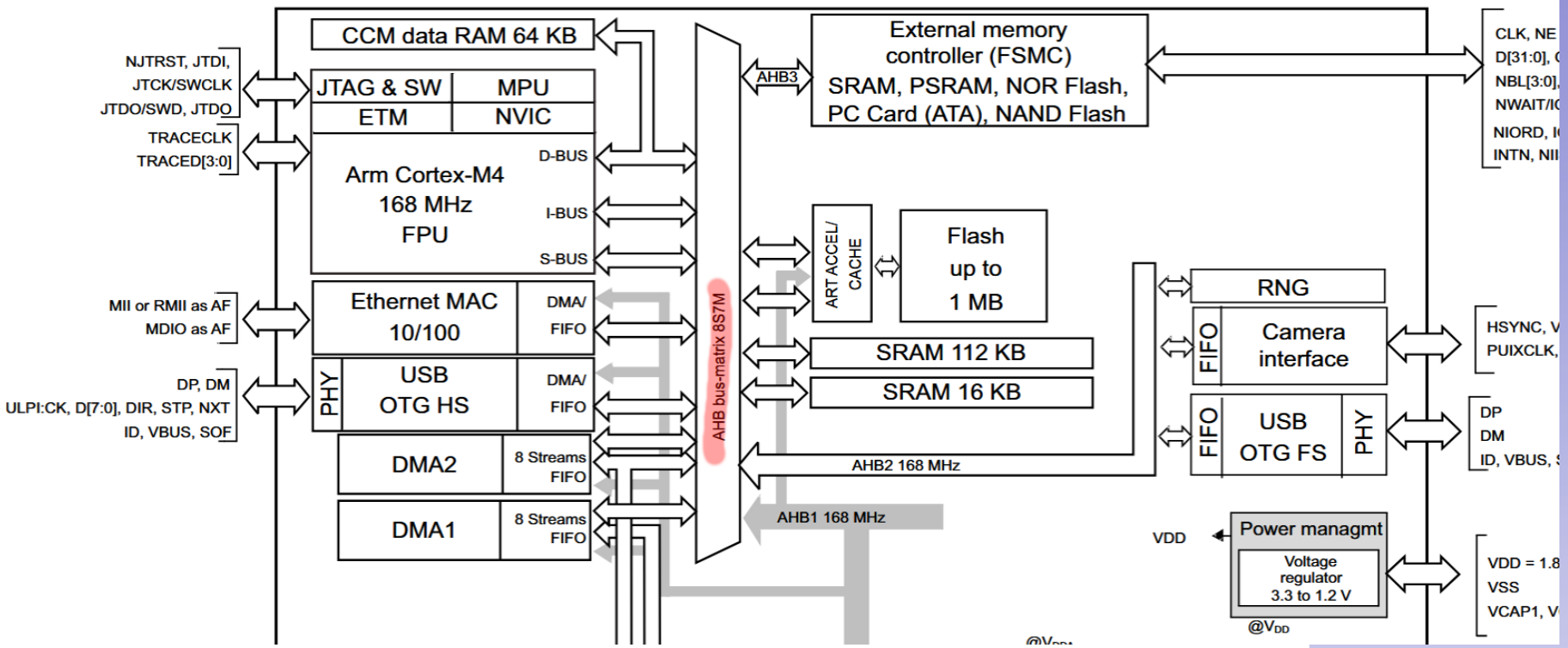
- При подключении нескольких устройств (несколько CPU, DMA и тд) и множества периферических устройств к одной шине вызывает большие задержки (и вообще говоря снижение скорости работы, а не повышение), т.к. в один момент времени шиной может пользоваться только одно устройство



- Решение - Crossbar коммутатор. Он позволяет каждому устройству подключаться к разной периферии одновременно



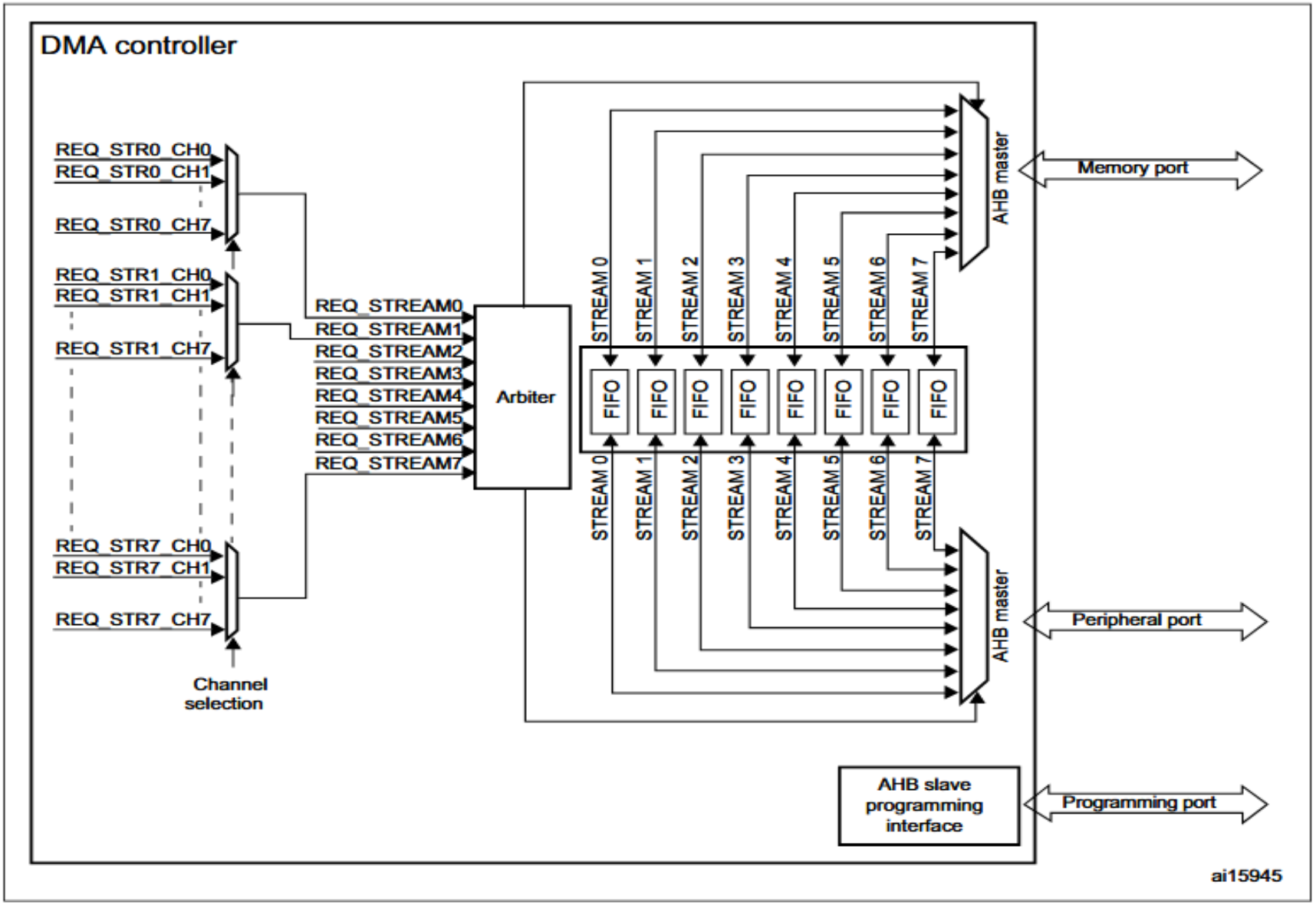
Внутреннее устройство контроллера DMA. AHB Matrix (19стр даташит)



- AHB Matrix - как раз такой коммутатор, позволяющий высокоскоростным устройствам МК пользоваться шинами AHB1,2 почти независимо.
- К ней подключены: ядро; DMA1,2; Ethernet; USB; CCM - Core-Coupled Memory - дополнительная оперативка.

Внутренне устройство DMA

Figure 32. DMA block diagram

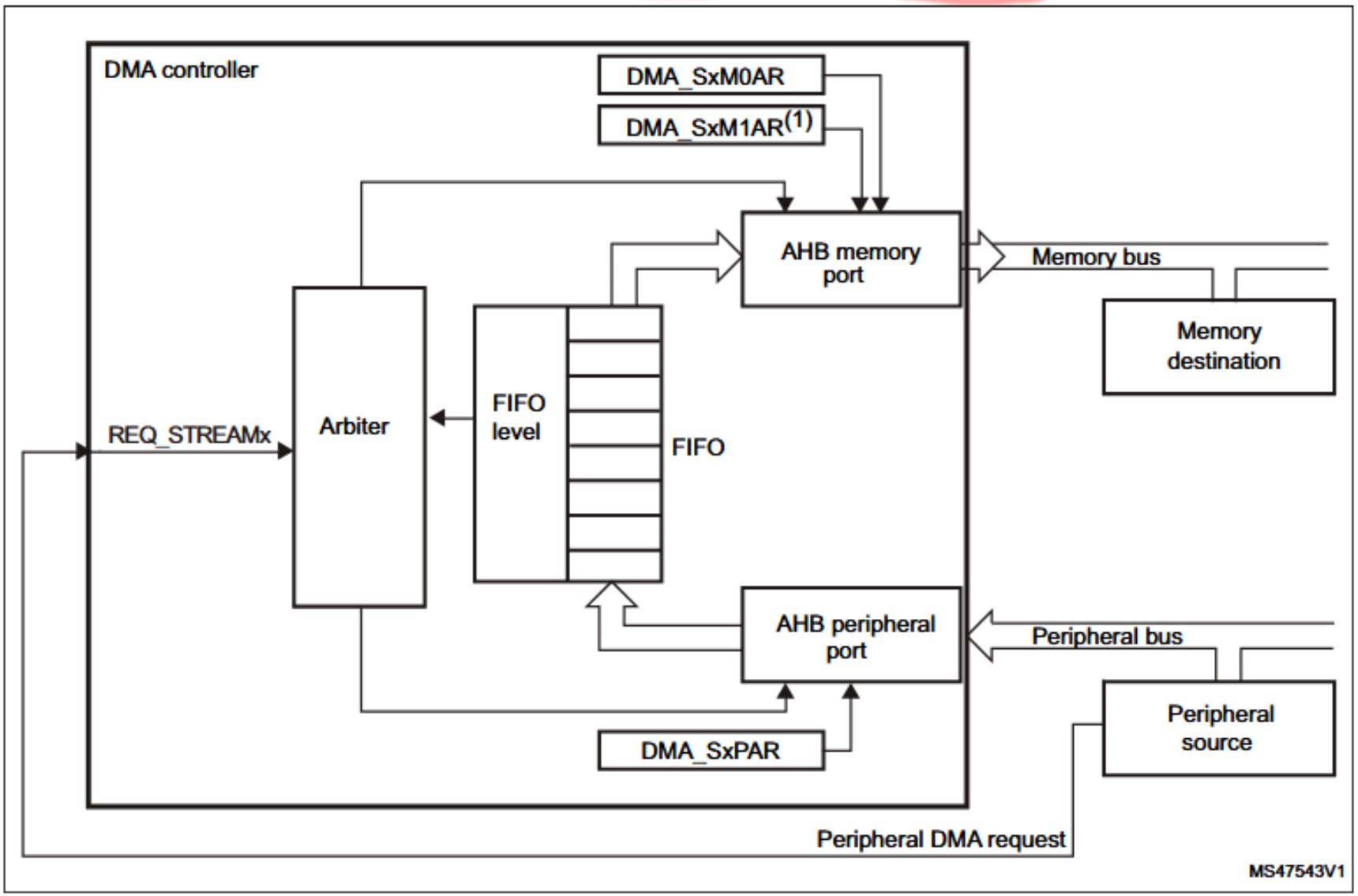


Внутренне устройство DMA

- Каждый контроллер DMA имеет по 8 потоков (stream). Каждый из них может работать с периферией независимо
- К каждому каналу подключен мультиплексор на выбор одного из 8 каналов передачи (channels)
- Пара: (stream n; channel k) полностью определяет, с каким устройством будет работать DMA.
- Потоки независимы, канал единственен на поток!
- Потоки входят в контроллер DMA(Arbiter). В нем происходит вся логика передачи данных
- Далее Арбитр связывает AHB Memory Port и AHB Peripheral Port с внутренним буфером FIFO.
- Отметим, что буферов FIFO ровно столько, сколько имеется потоков, т.е.8

Арбитр DMA

Figure 36. Peripheral-to-memory mode

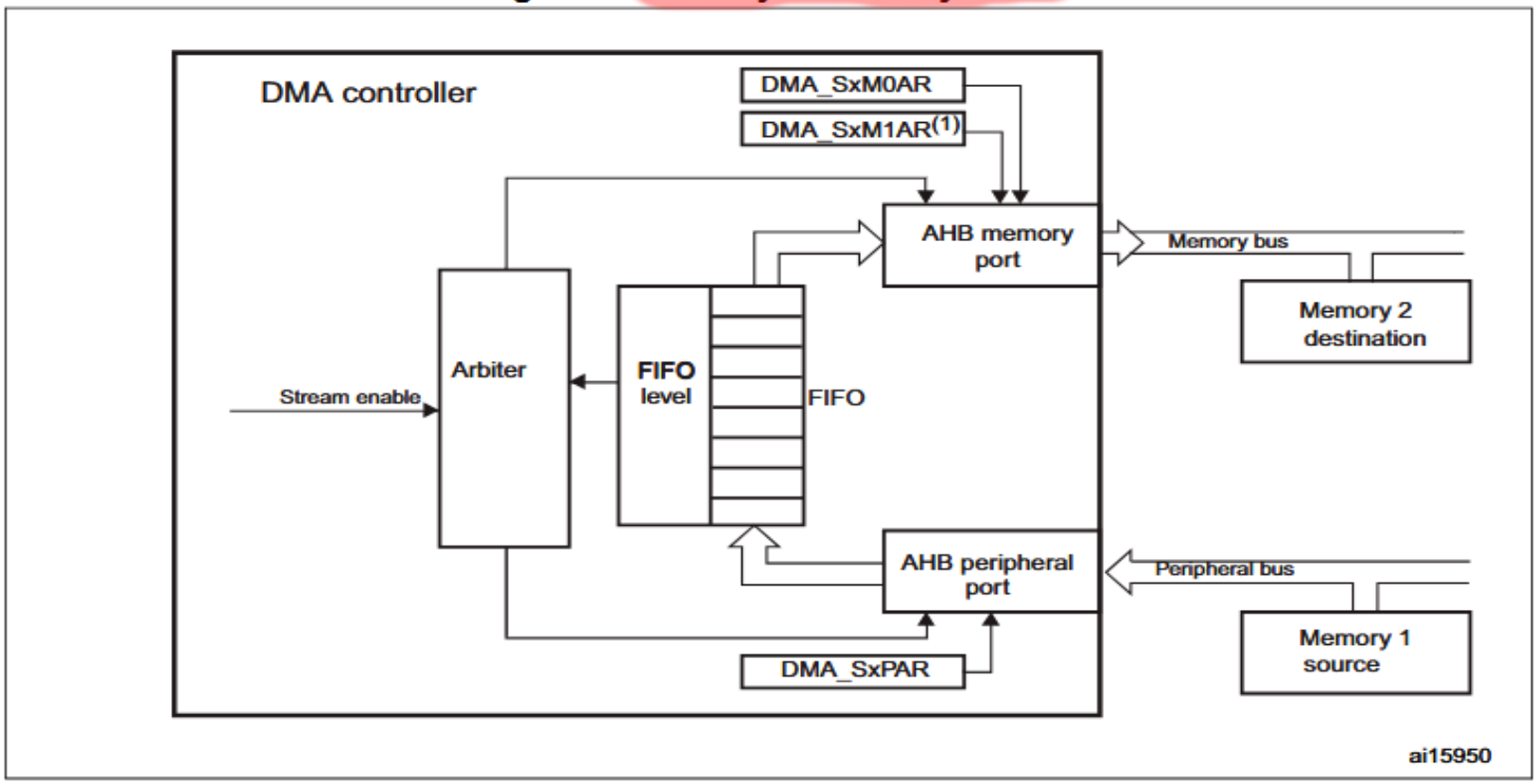


1. For double-buffer mode.

- Рассмотрим самого арбитра изнутри
- Как было видно на первой диаграмме, есть два порта для общения по шинам AHB: Memory port и Peripheral port.
- Они подключены к буферу FIFO - First Input First Output, т.е. он организован по приницпу очереди. Этот буфер используется в случае работы с медленной периферией. Данные накапливаются а далее передаются сразу пачкой для оптимизации использования шины.
- FIFO level - счетчик числа пакетов в FIFO
- Сам арбитр управляет логикой передачи.
- Peripheral source связан с арбитром сигналом запроса передачи данных - REQ_STREAMx
- В другую сторону (Память $\Longrightarrow$ периферия) передача аналогична

- Peripheral Port может использоваться для общения с памятью (в конце концов периферия и память находятся на одной шине и, значит, в одном адресном пространстве). Это нужно для копирования данных в памяти.
  Отметим, что тогда этот порт менее эффективно работает

Figure 38. Memory-to-memory mode

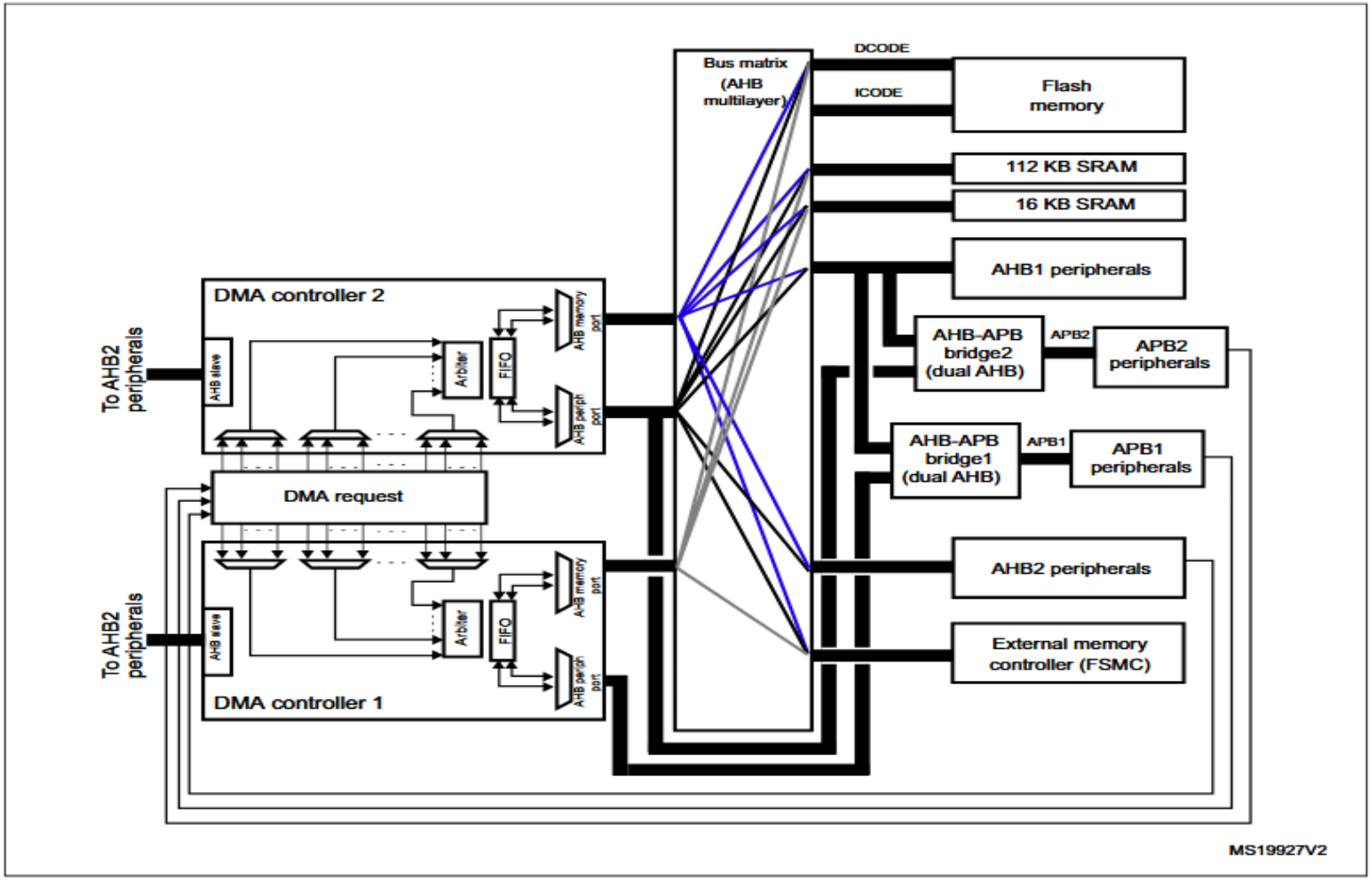


1. For double-buffer mode.

Отличие DMA 1 и 2

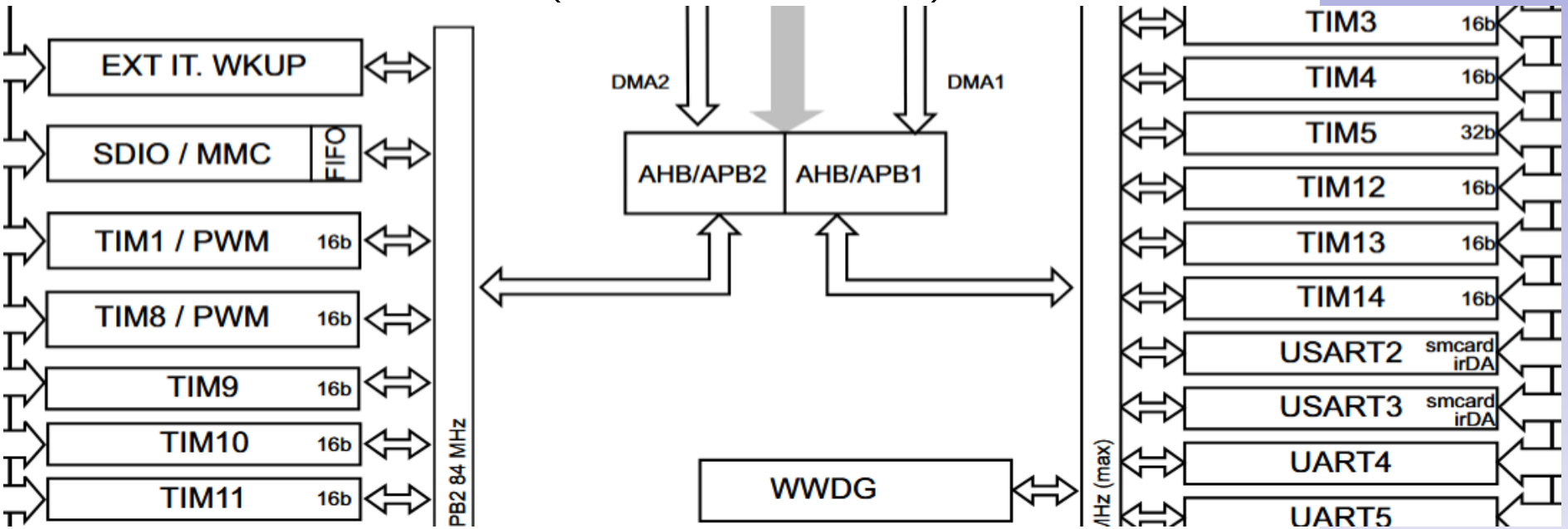
- Единственное отличие двух контроллеров - к чему они подключены внутри матрицы AHB

Figure 33. System implementation of the two DMA controllers (STM32F405xx/07xx and STM32F415xx/17xx)



1. The DMA1 controller AHB peripheral port is not connected to the bus matrix like DMA2 controller. As a result, only DMA2 streams are able to perform memory-to-memory transfers.

• Только у DMA 2 Peripheral port подключен к памяти. Поэтому только DMA2 может заниматься копированием данных в другие сегменты памяти (что и указано в сноске). • Отметим, что у DMA 1 Peripheral Port вообще не подключен к AHB Matrix, а сразу идет на AHB-APB bridge. У DMA 2 он подключен и к матрице, и к мосту. • AHB-APB bridge - узел, соединяющий быструю шину AHB 1 с медленными APB 1, 2 (стр 19 даташит):



Код

Программа лежит в репозитории, ничего нового там нет. Напишем еще один класс.

•Для определения набора (DMA1,2; stream 0-7, channel 0-7) используйте reference manual, главу про DMA, страницы 307-308. Разберем параметры:

- DMA_PeripheralBaseAddr - адрес регистра для взаимодействия (регистры такие же ячейки памяти, так что тоже имеют адрес), например (uint32_t) & USART2->DR
- DMA_Memory0BaseAddr - такой же адрес для памяти
- DMA_DIR - направление передачи данных
- DMA_BufferSize - количество данных для передачи
- DMA_MemoryInc - нужно ли смещать указатель на ячейку порта памяти после каждого пакета. Например удобно при заполнении массива

- DMA_PeripheralInc - то же самое, но на порту периферии (при копировании память $\Longrightarrow$ память это удобно, если же порт подключен к периферии то он будет считывать рандомные регистры в системном порядке).
- DMA_PeripheralDataSize и DMA_MemoryDataSize - размеры передаваемых данных, должны совпадать(быть не меньше чем)с размерами регистров периферии.
- DMA_Mode - режим работы. При режиме Normal - после передачи пакетов в количестве buffer size - останавливает работу. Circular - сбрасывает счетчик (memory inc) и начинает по новой сразу.
- DMA_Priority - приоритет потока внутри DMA.
- DMA_FIFOMode - режим буфера FIFO. disable - передача каждого пакета сразу. Enable - накопление пакетов в FIFO и передача сразу нескольких.

- DMA_FIFOThreshold - порог заполения буфера FIFO после которого начнется передача
- DMA_MemoryBusrst и DMA_PeripheralBurst - Настройка «взрывной» передачи (только если включен FIFO).
  Если выключен - пакеты передаются последовательно. В промежутках между ними CPU может перехватывать шину. INC4,8,16 - передача пакетов пачкой по 4,8,16 штук соответственно.

пример использования

https://github.com/haaroner/Comma_2021/blob/master/src/dma/dma.cpp

https://github.com/haaroner/Comma_2021/blob/master/src/adc/adc.cpp

Вызов - в main файле.

Пример для нашего кода(гптшный ☺):

```
uint8_t txBuffer[32] = "Hello DMA from STM32F4!";

int main(void) {
    // На STM32F407 UART2_TX сидит на DMA1, Stream 6, Channel 4
    DMA_Controller uartDma(DMA1_Stream6, DMA_Channel_4);

    // Инициализируем передачу Memory -> Peripheral
    uartDma.InitM2P((uint32_t)txBuffer, (uint32_t)&USART2->DR, 32);

    // Запускаем
    uartDma.Start();

    while (1) {
        // Основной цикл
    }
}
```